Trabajo 1. Organización de ficheros

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Administración de Bases de Datos

Profesor: Ignacio José Blanco Medina

Fecha: 25 de marzo de 2026

Índice General

1 Introducción **1**

2 Estructuras soporte **1**

3 TDA fileRecord **2**
3.1 Estructura en memoria 2
3.2 Operaciones públicas 3
3.3 Operaciones privadas 4

4 TDA fileBlock **5**
4.1 Estructura en memoria 5
4.2 Operaciones públicas 5
4.3 Operaciones privadas 8

5 TDA fileHeader **9**

6 TDA fileASF **9**
6.1 Estructura en memoria 10
6.2 Operaciones privadas 10
6.3 Operaciones públicas 16

7 Resumen de operaciones físicas y de acceso a disco **21**

1 Introducción

A lo largo de este documento se va a realizar el trabajo 1 de la asignatura de *Administración de Bases de Datos*. Dicho trabajo consiste en documentar, diseñar y especificar algunos tipos de datos abstractos para el trabajo con ficheros de carácter secuencial físico.

Estos tipos de datos abstractos están basados en la estructura en memoria de los ficheros de un sistema operativo; que como vimos en clase, no son más que una secuencia de bloques junto a una cabecera, donde cada bloque es una secuencia de registros precedidos globalmente por una cabecera.

De cada uno de ellos, trataremos la *estructura en memoria*, las *operaciones públicas* y las *operaciones privadas* [1]. Por tanto, debemos especificar y diseñar los tipos de datos abstractos **TDA fileASF**, **TDA fileHeader**, **TDA fileBlock** y **TDA fileRecord**.

2 Estructuras soporte

Con vistas a trabajar con mayor facilidad vamos a definir una serie de estructuras que trataremos como una sola. Esto es causa de que son estructuras unificadas para todos los objetos de un mismo tipo de dato abstracto.

Estas estructuras serán dos:

1. *unifiedRecord*: esta estrucutra refleja el conjunto de atributos del tipo de dato abstracto *TDA fileRecord* que están unificados para todos los objetos del mismo.

   Esta estructura está compuesta por los atributos *headerRecordSize*, *numAttr*, *tipoCampos* y *camposSize*; es decir, es la siguiente

```
private:
estructura unifiedRecord {
    entero headerRecordSize;
    entero numAttr;
    secuencia<entero> tipoCampos;
    secuencia<entero> camposSize;
}
```

Listing 1: Estructura *unifiedRecord*

2. *unifiedBlock*: esta estrucutra refleja el conjunto de atributos del tipo de dato abstracto *TDA fileASF* que modelan ciertas características de todos los bloques del fichero, en su uso serán constantes y servirán para el desplazamiento del manejador de fichero.

   Esta estructura está compuesta por los atributos *headerBlockSize* y *blockSize*; es decir, es la siguiente

[1] Todo ello en pseudocódigo

```
private:
estructura unifiedBlock {
    entero headerBlockSize;
    entero blockSize;
}
```

Listing 2: Estructura *unifiedBlock*

Cabe destacar que estas estructuras no son estrictamente necesarias, simplemente se han creado para facilidad de comprensión. Entenderemos que los operadores de igualdad hacen copia parámetro a parámetro; así como que su creación se hace atributo por atributo, siempre que la visibilidad del objeto en el que se encuentra lo permita.

Además, debo comentar que, visto como estructura sin más, los atributos no son constantes; sin embargo, una vez formen parte de alguno de los tipos de datos abstractos a desarrollar sí pasarán a ser constantes.

Estas estructuras ayudarán a comprender que, en la situación en la que estamos (bloques homogéneos con registros de longitud fija), esta estructura se impone al principio de forma conjunta y no se puede manipular.

3 TDA fileRecord

Comenzamos por el más interno, lo que ayudará a aprender de forma constructiva el objetivo de la práctica.

3.1 Estructura en memoria

Siguiendo el guión de la práctica, un registro es una secuencia de bytes que almacena dos aspectos, el primero de ellos es la estructura propia del registro y el segundo son los datos siguiendo la estructura que se ha impuesto en el primer dato.

```
// Representación en memoria del registro
private:
estructura fileRecord {
    const unifiedRecord structRecord;
    secuencia<bytes> content;
}
```

Listing 3: Especificación de la estructura del registro en memoria

Como vemos, el atributo *structRecord* que almacena toda la estructura del registro y que es idéntica en todos los de su mismo tipo, es constante. Esto pone de manifiesto que se trabajará como un todo y que no podrá modificarse una vez creado el objeto.

Por añadidura, es conveniente recordar que esta estructura vendrá reflejada en la cabecera del fichero y deberán heredarla del mismo al ser creados para mantener la estructura homogénea de los bloques con registros de longitud fija.

3.2 Operaciones públicas

Puesto que uno de los requisitos que se nos impone en la práctica es que el contenido de los registros pueda ser obtenido por un objeto de tipo *TDA fileASF*, debemos imponer algunas de las funciones de tipo *get* y tipo *set*.

```
// Getter y Setters
public:
/**
* Getter para obtener la estructura del registro
*
* @ret: copia de la estructura del registro
**/
unifiedRecord getStructRecord() const;

/**
* Getter para obtener el contenido de todo el registro
*
* @ret: copia del contenido del registro
**/
secuencia<bytes> getContent() const;

/**
* Setter para cambiar el contenido de todo el registro
*
* @param newContent: secuencia de bytes que alberga el nuevo
    contenido del registro
*
* @pre: content.size()==newContent.size
* @pos: No se cambia el tamaño del registro, pero sí su contenido
**/
void setContent(secuencia<bytes> newContent);
```

Listing 4: Operaciones públicas del registro

Con respecto al único *setter* que he creado, he supuesto que los datos del nuevo contenido tienen el mismo tipo; aún así, puesto que trabajamos con secuencias de bytes, no es un problema, simplemente los valores se interpretarían de forma distinta.

Realmente, ese *setter* no es correcto al completo, en una estructura del tipo que trabajamos la modificación de los registros está prohibidísima [2]; sin embargo, como tipo de dato abstracto puede resultar útil.

Como aclaración, el motivo por el cual todos los *getter* devuelven una copia es para evitar posibles modificaciones por referencia dependiendo del lenguaje de programación que se use; verificando así que no se pueden modificar datos que no se deban modificar.

Por último, debemos crear un constructor y un destructor de este tipo de dato.

```
// Constructor y destructor del registro
public:
/**
```

[2] Se marca como borrado y se crea otro con la misma clave y los cambios necesarios.

```
* Crea una instancia del tipo fileRecord con los datos pasados como
    argumentos
* @param structRecord: estructura del registro a crear
* @param content: contenido de cada uno de los campos del registro
*
* @pre: structRecord.headerSize>0, structRecord.numAttr>0
* @pos: Se crea una instancia de tipo fileRecord
**/
fileRecord (unifiedRecord structRecord, secuencia<bytes> content);

/**
* Constructor de copia
* @param otherFileRecord: registro a copiar
*
* @pre: otherFileRecord != null
* @pos: Se crea una instancia de tipo fileRecord idéntica a
   otherFileRecord
**/
fileRecord (fileHeader otherFileRecord);

/**
* Destruye una instancia de fileRecord
*
**/
~fileRecord()
```

Listing 5: Constructor y destructor del registro

3.3 Operaciones privadas

Como cabía de esperar, considero que este tipo de dato no admite operaciones privadas debido a que no dispone de una funcionalidad activa. Es decir, es claro que la principal funcionalidad de este tipo de dato es portar información de memoria a disco y de disco a memoria, por lo que es pasiva.

Por tanto, desde mi punto de vista, las operaciones privadas exigidas para el manejo del disco se delegan al tipo de dato abstacto *TDA fileASF*.

No obstante, podríamos pensar en una operación privada que realice una de las comprobaciones más importantes, la verificación de tamaños en la creación de una instancia de este tipo de dato; tendría la siguiente especificación:

```
// Operaciones privadas
private:
/**
* Comprueba las restricciones de que los tamaños y cantidades sean
   positivos así como que el tamaño del contenido sea igual a la
   suma de los tamaños especificados en camposSize al estar en un
   ASF. Además verifica que haya tantos tipos de campos como numero
    de atributos especificados. En definitiva, realiza las
   comprobaciones necesarias para no crear un objeto inconsistente.
*
* @param structRecord: estructura del registro
* @param cont: contenido de cada uno de los campos del registro
*
```

```
* @ret: true -> tamaños positivos y dentro de las restricciones de
    tipos
**/
boolean checkSizes (unifiedRecord structRecord, secuencia<bytes>
    content);
```

Listing 6: Operaciones privadas del registro

4 TDA fileBlock

Está claro que un fichero es un conjunto de registros. Sin embargo, la memoria no trabaja con registros como tal, sino que trabaja con bloques, cada uno de ellos compuesto por un conjunto de registros.

4.1 Estructura en memoria

Por tanto, siguiendo la especificación sobre la recodificación de las cabeceras y bloques al leer del disco, la estructura del bloque no será más que una secuencia de bytes que contiene todo lo especificado en la descripción de la misma dentro de la descripción de *TDA fileASF* del guión del trabajo.

```
private:
estructura fileBlock {
    secuencia<bytes> bloque;
}
```

Listing 7: Especificación de la estructura del bloque del fichero en memoria

Los accesos a las partes del bloque especificadas en el guión deberán realizarse mediante *offsets* dependiendo del tamaño de los campos del registro especificado en la estructura *unifiedRecord* del propio fichero al que pertenece.

Además, como el fichero dispone de la estructura de bloque, se podrá conocer el tamaño de la cabecera de bloque para poder realizar el último *offset* que quedaba por determinar.

Por añadidura, ya que cada elemento de la secuencia de registros del *fileBlock* es una cabecera de registro formada por dos enteros y el contenido como secuencia de bytes, supondremos que podemos acceder al contenido de cada uno aplicando un *offset* de 8 bytes; así como a cada uno de los elementos de la cabecera con un *offset* de 0 y 4 bytes respectivamente.

4.2 Operaciones públicas

Como todo tipo de dato abstracto, deberemos poder crearlo y destruirlo; puesto que en ocasiones necesitaremos crear un bloque cuyo contenido sea vacío, he decidido permitir esta posibilidad.

```
// Constructor y destructor de bloque
public:
/**
```

```
* Crea una instancia del tipo fileBlock con los datos pasados como argumentos
* @param content: registros del bloque
*
* @pre: (content-headerSize)/recordSize<=Bfr
* @pos: Se crea una instancia de tipo fileBlock
**/
fileBlock (secuencia<bytes> content);

/**
* Crea una instancia de tipo fileBlock vacía, solo con el idBlock pasado como argumento.
* @param idBlock: identificador del bloque
*
* @pre: idBlock>0
* @pos: Se crea una instancia de tipo fileBlock vacía
**/
fileBlock (entero idBlock);

/**
* Destruye una instancia de fileRecord
*
* @pos: Se crea una instancia de tipo fileRecord
**/
~fileBlock()
```

Listing 8: Constructor y destructor de bloque

Solo con estas operaciones, nuestro tipo de dato *fileBlock* es un bolso con cartas pero no permite trabajar con ellas, ni obtenerlas, ni añadirlas, ni, tampoco, eliminarlas.

Para ello, crearemos algunas operaciones que hagan este tipo de operaciones. Cabe destacar que, debido a la estructura que manejamos, no debemos permitir el modificado de los registros [3] pues no será más que un borrado y un añadido.

Cada una de estas operaciones actúa de la siguiente manera:

- *addRecord*: se encarga de añadir un registro al final del bloque; podremos avanzar en el bloque gracias a que los registros son de longitud fija y conoceremos el factor de bloqueo.
- *deleteRecord*: independientemente de la versión, se encarga de buscar el registro comenzando desde el principio hasta el final; si encuentra el registro modifica el estado de registro a borrado.

Las operaciones quedan como sigue:

```
// Operaciones públicas de bloque
public:
/**
* Añade un registro al bloque; en caso de que el bloque esté completo devolverá -1
*
```

[3] En *fileRecord* sí lo habíamos permitido; sin embargo, comenzamos ya a ceñirnos a la estructura que vamos a desarrollar.

```
* @param record: registro a añadir
* @param structRecordFile: estructura de los registros del fichero
*
* @pre record.structRecord = structRecordFile
* @pos: se añade el registro al bloque si es posible por tamaño, el regisro dispondrá de su identificador de registro y de estado a true.
*
* @ret: entero que determina el estado de la operación:
*       - (-1) el bloque está completo y se debería crear otro bloque
*       - (0) no se ha podido realizar la operación, pero el bloque dispone de espacio.
*       - (1) la operación se ha realizado correctamente
*/

entero addRecord(fileRecord record, unifiedRecord structRecordFile);


/**
* Borra un registro del bloque
*
* @param idRecord: identificador del registro a borrar
*
* @pre: idRecord > 0
* @pos: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(entero idRecord);

/**
* Borra un registro del bloque por clave
*
* @param clave: clave del registro a borrar
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: clave.type==structFileRecord.tipoCampos[clave]
* @pos: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(secuencia<bytes> clave, unifiedRecord structFileRecord);

/**
* Borra un registro del bloque que sea idéntico al pasado como argumento
*
```

```
* @param record: registro a borrar
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: record.unifiedRecord==structFileRecord
* @pos: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(fileRecord record, unifiedRecord structFileRecord);
```

Listing 9: Operaciones públicas de bloque

4.3 Operaciones privadas

Para que muchas de las operaciones tengan sentido, debemos volver al tema del desplazamiento sobre el bloque; tal y como dijimos en la sección 4.1 debemos manejarnos con *offsets* que, gracias al fichero en el que se encontraría el bloque sería posible, en virtud de la estructura *unifiedRecord*.

```
// operaciones privadas de bloque
private

/**
* Busca un registro por identificador de registro
*
* @param idRecord: identificador de registro
* @param structuRecord: estructura del registro para poder realizar desplazamientos
*
* @pre: idRecord<Bfr
* @pos: simplemente busca hacia delante, nunca retrocede.
* @ret: copia del registro encontrado, o en caso contrario, un registro vacío con identificador -1.
*/
fileRecord find(entero idRecord, unifiedRecord structRecord) const;

/**
* Busca un registro por clave de registro
*
* @param clave: clave del registro
* @param structuRecord: estructura del registro para poder realizar desplazamientos
*
* @pos: simplemente busca hacia delante, nunca retrocede.
* @ret: copia del registro encontrado, o en caso contrario, un registro vacío con identificador -1.
*/
fileRecord find(secuencia<bytes> clave, unifiedRecord structRecord) const;
```

Listing 10: Operaciones privadas de bloque

5 TDA fileHeader

En principio, este tipo de dato no es necesario y, de hecho, sería desconocido para el usuario. El motivo de ello es el siguiente; cuando trabajamos con un fichero, la cabecera dispone de todos los datos del fichero inherentes al propio fichero y que son exclusivos de él. Por tanto, no tiene sentido crear un tipo de dato abstracto para la cabecera puesto que sus operaciones dependerán de las del fichero en sí.

Aún así, voy a crear una estructura de datos que me ayudará a trabajar con ella de forma más compacta en el modelaje del *TDA fileASF*; la llamaré *fileHeader* y su gestión se realizará en el tipo de dato que la contiene.

```
// Representación en memoria de una cabecera de fichero
estructura fileHeader {
    secuencia<bytes> content;
    const pointer <const entero> tamHeaderpt;
    const pointer <entero> numRecordspt;
    const pointer <entero> numBlockspt;
    const pointer <const unifiedRecord> structRecordpt;
    const pointer <const unifiedBlock> structBlockpt;
}
```

Listing 11: Especificación de la estructura de otra forma de la cabecera del fichero en memoria

Como podemos ver, debido a que no debemos codificar el contenido de la cabecera del fichero, tendremos una secuencia de bytes que guarda el contenido de la cabecera. El motivo por el que disponemos de punteros a la posición de memoria de cada uno de los elementos de la cabecera reside en la facilidad de gestión de la misma.

Dichos punteros conocen el tipo de dato al que apuntan; por tanto, al consultarlos, recibiremos exactamente la cantidad de bytes necesarios por cada tipo de dato, es decir, es similar a un desplazamiento por *offsets*.

Sin embargo, este método tiene la comodidad de que, una vez se establecen los punteros al crear el tipo de dato *TDA fileASF* indicando por única vez los *offsets* correspondientes, no necesitaremos realizar esos cálculos ninguna vez más.

Como es de esperar, los punteros que apuntan a enteros agruparán 4 bytes en su lectura, el puntero que apuntan a la estructura de tipo *unifiedRecord* agrupará un tamaño variable pero conocido en la apertura del fichero y lectura de la cabecera; y , por último, el puntero que apunta a la estructura de tipo *unifiedBlock* agrupará 8 bytes de contenido.

Por último, cabe recalcar que, aunque esté tratando los elementos a los que apuntan los punteros como objetos predefinidos, realmente se tratan como bytes y sólo cuando se usan de forma independiente son decodificados para su manejo.

6 TDA fileASF

Estamos ya en disposición de modelar el tipo de dato abstracto *fileASF*, nuestro objetivo. Para ello, usaremos las estructuras trabajadas previamente.

6.1 Estructura en memoria

La estructura en memoria que se nos plantea en el guión es la siguiente:

```
// Representación en memoria de un fichero ASF
private:
estructura fileASF {
    secuencia<caracter> diskFilename;
    diskFile fileManager;
    fileHeader header;
    fileBlock chargedBlock;
    boolean changedHeader;
    boolean changedContent;
    entero numRecordsLastBlock;
}
```

Listing 12: Especificación de la estructura del fichero en memoria

Realmente, hay una pequeña modificación; esta es *numRecordsLastBlock* que representa el número de bloques que tiene el último bloque del fichero. Este atributo es una mejora, permite conocer en qué ocasión vamos a tener que añadir un bloque cuando añadamos o modifiquemos un registro.

El cálculo para esta gestión es sencillo y simplemente depende del factor de bloqueo; eso sí, debemos gestionar otro atributo más. Aún así, creo que es una mejora considerable.

6.2 Operaciones privadas

A diferencia del tipo de dato abstracto *fileBlock*, considero que es mejor definir las operaciones privadas primero; ayudará a comprender la construcción que haremos de las operaciones públicas.

Resaltaré algunas de ellas:

- *fCloseAndOpen*: se encarga de cerrar y abrir el fichero para colocar el manejador de fichero en la posición inicial; es decir, presenta una utilidad bastante grande a la hora de buscar. Claramente, delega en *fClose* y *fOpen*, pero simplemente no necesita la ruta del archivo como argumento, usa *diskFilename*.
- *chargeNextBlock*: es la encargada de cargar el siguiente bloque del fichero; para ello, debe escribir el bloque que hay cargado en disco para cargar el siguiente. Es una función útil que sirve para recorrer el fichero completo al reordenar.
- *copyOnFile*: tiene la obligación de añadir los registros que se pasan como argumento al fichero que se pasa como argumento; es una función exclusivamente creada para realizar la reordenación bloque a bloque.
- *goToLastBlock*: como dice su nombre, mueve el manejador de fichero a la primera posición del último bloque del fichero y lo carga en *chargedBlock*. Previamente, se ha escrito el bloque que había cargado en disco.

  En caso de identificarse que el bloque que hay cargado ya es el último, no se realiza la operación.

Si nos fijamos, en todas o casi todas las operaciones he considerado trabajar con un código de estado de la operación. La causa viene impuesta por la gestión de inconsistencias; considero que, si tratamos de realizar la gestión de la inconsistencia justo cuando ocurre, es más sencillo mantener el fichero de forma correcta.

```
// Operaciones privadas de fileASF
private:

/**
* Obtiene un manejador de fichero para el fichero pasado como argumento
*
* @param path: ruta del fichero que se quiere abrir.
*
* @pre: path debe existir en el sistema, en caso contrario da un error
* @pos: this.diskFilename = path
*        el fichero no es modificado aunque puede ser modificado con posterioridad.
*
* @ret: devuelve un diskFile que apunta a la primera posición de memoria del fichero.
*/
diskFile fOpen(secuencia<caracter> path);

/**
* Cierra el fichero actual liberando la memoria dinámica usada.
*
* @pos: this.fileManager=nullptr
*
* ret: codigo de operación:
*       (-1) no se ha podido cerrar el fichero o no existe
*       (0) el bloque de memoria o la cabecera están modificados
*       (1) se ha cerrado correctamente
*/
entero fClose();

/**
* Cierra y abre el fichero actual para comenzar al principio.
*
* @pos: this.diskFilename = path
*        el fichero no es modificado aunque puede ser modificado con posterioridad.
*        fileManager apunta a la primera posición de memoria del fichero.
* ret: codigo de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (0) el bloque de memoria o la cabecera están modificados
*       (1) se ha cerrado y abierto correctamente, fileManager está en posición correcta.
*/
entero fCloseAndOpen();

/**
* Devuelve una copia del registro que tiene la clave pasada como
```

```
argumento.
*
* @param clave: clave del registro que se busca
*
* @pre: clave tiene una estructura adecuada, siguiendo la que hay en header.structRecord.
* @pos: el registro podrá ser modificado.
*       Cierra y abre el fichero para comenzar a buscar desde el principio.
*       fileManager se desplaza a lo largo del fichero.
*
* ret: copia del registro que se ha pedido, en caso de no encontrarse se devuelve vacío.
*/
fileRecord findRecord(secuencia<bytes> clave);

/**
* Devuelve una copia del registro que es idéntico al pasado como argumento.
*
* @param record: registro que se busca.
*
* @pre: record tiene una estructura adecuada, siguiendo la que hay en header.structRecord.
* @pos: el registro podrá ser modificado.
*       Cierra y abre el fichero para comenzar a buscar desde el principio.
*       fileManager se desplaza a lo largo del fichero.
*
* ret: copia del registro que se ha pedido, en caso de no encontrarse se devuelve vacío.
*/
fileRecord findRecord(fileRecord record);

/**
* Escribe el contenido del bloque en memoria.
*
* @pre: changedContent==true;
* @pos: el bloque queda almacenado en memoria poniendo el flag changedContent a false.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido escribir el bloque en memoria o hay alguna inconsistencia con la cabecera.
*       (0) el bloque no ha sido modificado.
*       (1) operación realizada correctamente.
*/
entero writeBlockOnDisk();

/**
 * Lee la cabecera del fichero desde el disco y la carga en memoria.
 *
 * @pos: Se cierra y abre el fichero para leer en la cabecera.
 *       actualiza los punteros si es necesario, si no están inicializados los inicialza correctamente.
```

```
 *        el contenido de la cabecera se carga en header.content
 *        el flag changedHeader se establece a false.
 *
 *
 * @ret: código de estado de la operación
 *        (-1) no se ha podido leer la cabecera o el fichero está inaccesible.
 *        (1) operación realizada correctamente.
 */
entero readHeaderFromDisk() ;

/**
 * Lee un bloque del fichero del disco y lo carga en memoria.
 *
 * @param idBlock: identificador del bloque que se desea leer.
 *
 * @pre: idBlock >= 0 y idBlock < header.numBlocks ;
 * @pos: si el bloque está después del bloque actual, se avanza el fileManager; en caso contrario, debe cerrarse y abrirse el fichero para después avanzar al bloque necesitado.
 *        el atributo chargedBlock se actualiza con la secuencia de bytes leída.
 *        el flag changedContent se establece a false.
 *
 * @ret: código de estado de la operación
 *        (-1) no se ha podido leer el bloque o el identificador está fuera de rango.
 *        (1) operación realizada correctamente.
 */
entero readBlockFromDisk(entero idBlock) ;

/**
* Escribe el contenido de la cabecera en memoria.
*
* @pre: changedHeader==true;
* @pos: Se cierra y abre el fichero para poder realizar la escritura en la cabecera.
*        changedHeader=false;
* @ret: codigo de estado de la operación
*        (-1) no se ha podido escribir la cabecera en memoria o hay alguna inconsistencia con el contenido.
*        (0) la cabecera no ha sido modificada.
*        (1) operación realizada correctamente.
*/
entero writeHeaderOnDisk();

/**
* Escribe todo el objeto en memoria.
*
* @pre: changedHeader==true and changedContent==true;
* @pos: delega en writeHeaderOnDisk y en writeBlockOnDisk
*
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación o hay alguna inconsistencia.
*        (0) ningún objeto ha sido modificado
```

```
*        (1) operación realizada correctamente.
*/
entero writeOnDisk();

/**
* Añade un registro a la cabecera
*
* @pos: changedHeader=true y header.numRecords+=1.
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación.
*        (1) operación realizada correctamente.
*/
entero addRecordOnHeader();

/**
* Sustrae un registro a la cabecera
*
* @pos: changedHeader=true y header.numRecords-=1.
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación.
*        (1) operación realizada correctamente.
*/
entero deleteRecordOnHeader();

/**
* Añade un bloque a la cabecera
*
* @pos: changedHeader=true
*        header.numBlocks+=1.
*
* @ret: codigo de estado de la operación
*        (-1) no se ha podido realizar la operación.
*        (1) operación realizada correctamente.
*/
entero addBlockOnHeader();

/**
* Obtiene una lista con una copia de todos los registros del bloque
    chargedBlock que no están modificados.
*
* @ret: secuencia de fileRecord con los registros no borrados de
    chargedBlock
*/
secuencia<fileRecord> getNonDeleted() const;

/**
* Copia la lista de registros pasada como argumento al final del
    fichero pasado como argumento.
*
* @param recordSequence: secuencia de registros a copiar
* @param newFile: fichero en el que copiarlos
*
* @pos: newFile cambia su contenido.
*/
void copyOnFile(secuencia<fileRecord> recordSequence, fileASF
    newFile);
```

```
/**
* Escribe el bloque chargedBlock en memoria si ha sido modificado y hay consistencia. Y carga el siguiente bloque (siguiente identificador) en chargedBlock.
* Si hay que cargar el primer bloque, fileManager avanza al primer bloque (offset=hedaer.tamHeaderpt* bytes) y lo carga, terminando apuntando al siguiente bloque.
*
* @pos: el contenido que había en chargedBlock se escribe en memoria.
*       chargedBlock tiene el contenido del siguiente bloque en memoria.
*       fileManager apunta a la primera posición de memoria del siguiente bloque
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (0) ya es el último bloque se ha creado un bloque nuevo y se ha cargado en chargedBlock.
*       (1) operación realizada correctamente.
*/
entero chargeNextBlock();

/**
* Avanza hasta el último bloque y lo carga en chargedBlock.
*
* @pos: se escribe el contenido de chargedBlock en disco antes de cargar el último bloque. Si está cargado el último bloque, no se escribe en disco.
*       se desplaza fileManager a la posición del último bloque donde se puede escribir.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (0) chargedBlock es el último bloque.
*       (1) operación realizada correctamente.
*/
entero goToLastBlock();
```

Listing 13: Operaciones privadas del fichero en memoria

Cabe destacar que el desplazamiento del manejador de fichero debe realizarse mediante *offsets*; el cálculo de los mismos dependerá de la estructura que queramos visitar o traspasar. Se siguieron las siguientes pautas:

- Si queremos avanzar un bloque de disco; el *offset* vendrá impuesto por los datos almacenados en *structRecord*, es decir, se avanzarán *headerBlockSize*+*blockSize* bytes.
- Si queremos avanzar al primer registro de un bloque; simplemente deberemos pasar la cabecera del mismo, es decir, se avanzarán *headerBlockSize* bytes.
- Si queremos avanzar un registro dentro de un bloque, deberemos añadir un *offset* dependiente de la suma de los tamaños de los campos reflejados en

structRecord.

- Si queremos avanzar a un campo de un registro concreto, deberemos realizar los cálculos del *offset* dependiendo de los tamaños de los campos reflejados en *structRecord*.

6.3 Operaciones públicas

Como sabemos, debemos disponer de un constructor y de un destructor, al menos. En nuestro caso, disponemos de un constructor que trata de abrir el fichero cuya ruta es la pasada como argumento y, en caso de no encontrarlo, lo crea.

La creación del objeto simplemente incluye la inicialización de la cabecera, la colocación del descriptor de fichero en la primera posición de memoria del primer bloque del fichero y deja vacío el bloque a cargar.

Además, claramente disponemos de un destructor que liberará la memoria dínamica utilizada y escribirá en disco el contenido del objeto en caso de modificación.

```
// Constructor y destructor
public:

/**
* Crea un objeto abriendo el fichero pasado como argumento o creá
    ndolo
*
* @pre: path debe ser una ruta de un sistema de ficheros.
* @pos: puede crear un fichero en el sistema de ficheros,
    completamente vacío.
*       abre el fichero pasado como argumento.
*       fileManager apunta a la primera posición de memoria del
    primer bloque del fichero.
*       chargedBlock está vacío hasta que se añada un registro.
*       se carga el contenido de la cabecera inicializando los
    punteros.
*
*/
fileASF(diskFilename path);

/**
* Destruye el objeto actual, escribiendo previamente en disco si
    hay modificaciones y son consistentes, en caso contrario, da un
    error.
*
* @pos: fileManager = nullptr
*       el objeto completo es destruido escribiendo en disco el
    contenido necesario.
*       libera la memoria dinámica utilizada.
*/
~fileASF();

}
```

Listing 14: Constructor y destructor del fichero

Antes de hablar sobre las operaciones públicas que le dan funcionalidad a este tipo de dato, es conveniente aclarar la diferencia entre *fOpen* y el *Constructor*. Ella reside en que el *Constructor* sí carga la cabecera del fichero cuando se crea el objeto, pues lo habitual será crear el objeto para trabajar con los registros. Sin embargo, *fOpen* sitúa el manejador de fichero en la primera posición del fichero, al ser una operación privada no es problemático y sí es útil para otro tipo de operaciones públicas como *writeHeaderOnDisk*.

Ahora sí, como operaciones públicas tendremos las encargadas de leer un registro, reordenar el fichero, insertar un registro, borrar un registro y modificar un registro. Tratemos un poco cada una de ellas:

- *getRecord*: simplemente cierra y abre el fichero para buscar el registro de forma secuencial, leyendo el fichero completo en caso de no encontrarlo. En caso de encontrarlo, devuelve el registro en sí y, si no lo encuentra, devuelve un registro vacío.

- *reorder*: su función es obtener un nuevo fichero que albergue a todos los registros del fichero actual que no estén borrados; para ello, hará uso de las funciones privadas *getNonDeleted* y *chargeNextBlock*.

  Además, se realiza la gestión de la cabecera a medida que se va realizando la copia de los registros en el nuevo fichero, añadiendo las cantidades sobre registros y bloques que sean requeridas, mediante la función *insertRecord* aplicada al nuevo fichero ya creado.

- *insertRecord*: Se encarga de cerrar y abrir el fichero, modificando la cabecera para reflejar que se va a añadir el registro. A continuación, si no podemos insertar en el último bloque por cumplir el factor de bloqueo (usa *numRecordsLastBlock* para determinarlo), aumenta el número de bloques reservados, desplaza el organizador de fichero al último bloque para insertar un nuevo bloque y añadir el registro. En caso de que se pueda añadir al final, simplemente se desplazaría el organizador de fichero al final último bloque para añadir el registro, habiendo escrito previamente en la cabecera el aumento del número de registros. Esta operación dispone de una operación análoga para una secuencia de registros, el comportamiento es parecido, simplemente las operacione sque se pueden hacer de forma conjunta se hace. [4]

- *deleteRecord*: es la encargada de borrar un registro de forma lógica; se busca el registro en el fichero, con las operaciones privadas que eso conlleva; en caso de que se haya encontrado, se modifica el contenido de la cabecera del fichero adecuadamente (sin disminuir la cantidad de bloques reservados) y, después, se realiza el borrado lógico del registro. Al igual que ocurría con la operación anterior, disponemos de una operación para secuencias de registros; la lógica de actuación sería conocer primero cúantos de ellos s pueden borrar para modificar la cabecera y después ir borrándolos uno a uno.

[4] Es decir, como podemos calcular los bloques que nos harán falta y sabemos los registros que añadiremos podremos modificar una sola vez la cabecera.

- *modifyRecord*: se encarga de modificar el contenido de un registro; tal y como hemos tratado ya, no podemos modificar como tal el contenido, sino que debemos borrar el registro a modificar y añadir un nuevo registro.

  En un inicio, deberemos saber si el registro está dentro del fichero; para ello, lo buscaremos y, en caso de que sí exista realizaremos la modificación.

  Supuesto que el registro existe en el fichero, vamos a añadir un registro; entonces, el factor de bloqueo(junto con *numRecordsLastBlock*) nos dirá si debemos añadir un nuevo bloque, que en caso de ser necesario, se reflejará en la cabecera de forma adecuada. A continuación, realizamos el borrado lógico del registro y añadimos uno al final del fichero con el contenido modificado, sin modificar la clave del registro inicial.

  Cabe destacar que, en caso de apagón, si hemos ido modificando la cabecera en cada una de las operaciones que hemos realizado, podremos intuir o recuperar la consistencia de forma correcta.

```
// Operaciones públicas
public:

/**
* Devuelve una copia del registro con la clave pasada como argumento.
*
* @param clave : clave del registro a buscar
*
* @pre: la estructura de la clave debe ser la adecuada para el fichero
*
* @ret: copia del registro encontrado, en caso de no encontrarlo devolverá un registro vacío.
*/
fileRecord getRecord(secuencia<caracter> clave);

/**
* Devuelve un fichero completamente nuevo cuyo contenido es la reordenación de este.
*
* @ret: fichero reordenado, si todos están borrados, crea un nuevo fichero que dispone de un bloque de memoria reservado y completamente vacío.
*/
fileASF reorder();

/**
* Inserta un registro al final del documento, en caso de necesitarlo, añade otro bloque al fichero.
*
* @pre: record.structRecord==this.header.structRecord
* @pos: se carga el último bloque en chargedBlock, escribiendo el que había en memoria.
*       this.header.numRecords+=1
*       changedHeader=true
*       changedContent=true
```

```
*        numRecordsLastBlock modifica dependiendo de la casuística:
*            - Si Bfr>numRecordsLastBlock -> numRecordsLastBlock+=1
*            - Si Bfr<=numRecordsLastBlock -> numRecordsLastBlock=1 y this.header.numBlocks+=1
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (1) se ha insertado correctamente.
*/
entero insertRecord(fileRecord record);

/**
* Inserta una secuencia de registros al final del documento, en caso de necesitarlo, añade los bloques necesarios al fichero.
*
* @pre: record[i].structRecord==this.header.structRecord
* @pos: se carga el último bloque en chargedBlock, escribiendo el que había en memoria de forma iterada.
*        this.header.numRecords es aumentado.
*        changedHeader=true
*        changedContent=true
*        numRecordsLastBlock y this.header.numBlocks se modifica siguiendo la lógica de la operación anterior.
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) algún/unos registros no se han podido insertar.
*        (1) se han insertado todos los registros correctamente.
*/
entero insertRecordSequence(sequence<fileRecord> records);

/**
* Realiza un borrado lógico del registro con la clave pasada como argumento.
*
* @pre: la clave dispone del formato y tipo adecuado para el fichero.
* @pos: se carga el bloque del registro en chargedBlock, escribiendo el que había en memoria.
*        this.header.numRecords-=1
*        changedContent=true
*        changedHeader=true
*        numRecordsLastBlock-=1
* @ret: código de operación:
*        (-1) no se ha podido realizar la operación o hay inconsistencias.
*        (0) el registro ya se encuentra borrado o no se ha encontrado.
*        (1) se ha borrado correctamente.
*/
entero deleteRecord(secuencia<caracter> clave);

/**
* Realizar un borrado lógico de una secuencia de registros con las claves pasadas como argumento.
*
```

```
* @pre: la claves disponen del formato y tipo adecuado para el fichero.
        claves.numElements>=this.header.numRecords
* @pos: se realizan las mismas modificaciones que en la operación anterior pero en escala.
* @ret: código de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (0) algún/unos de los registros ya se encuentran borrado o no se han encontrado.
*       (1) se han borrado todos correctamente.
*/
entero deleteRecordSequence(secuencia<secuencia<caracter>> claves);

/**
* Modifica el contenido de un registro sin cambiar su clave.
*
* @pre: record.structRecord==this.header.structRecord
* @pos: puede modificar la cabecera añadiendo (o no) un bloque (depende de la situación).
*       this.header.numRecords no se modifica.
*       la clave del registro no se permite su modificación.
* @ret: código de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (0) no se ha encontrado el registro o dispone del mismo contenido que el sugerido por la modificación.
*       (1) se ha modificado correctamente.
*/
entero modifyRecord(fileRecord record);

/**
* Modifica el contenido de una secuencia de registros sin cambiar sus claves.
*
* @pre: record[i].structRecord==this.header.structRecord
* @pos: puede modificar la cabecera añadiendo (o no) un(o varios) bloque/s (depende de la situación).
*       this.header.numRecords no se modifica.
*       las claves de los registros no son modificadas.
* @ret: código de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (0) no se ha encontrado algún/unos registro/s o dispone/n del mismo contenido que el sugerido por su modificación.
*       (1) se han modificado todos correctamente.
*/
entero modifyRecordSequence(secuencia<fileRecord> records);

}
```

Listing 15: Operaciones públicas del fichero

Es importante recalcar que, aunque no se han especificado completamente cómo son los desplazamientos necesarios de la estructura *fileManager*, si queremos movernos de registro en registro el *offset* será el tamaño del registro junto al tamaño de la

cabecera del registro y, en caso de querer movernos por bloques, el *offset* será el tamaño de bloque junto a la cabecera de bloque.

7 Resumen de operaciones físicas y de acceso a disco

Aunque ya se ha ido especificando en cada una de las operaciones, dejo una tabla resumen que trata qué accesos a disco presenta cada operación, si se escribe la cabecera y si hay aperturo o cierre del fichero.

Operación	**Lectura / Escritura y Modificaciones**	**Escribe Cabecera**	**Apertura / Cierre**
`readHeaderFromDisk` *(Privada)*	**Lectura.**	No	**Apertura** y **Cierre**
`writeHeaderOnDisk` *(Privada)*	**Escritura.**	**Sí**	**Apertura** y **Cierre**
`readBlockFromDisk` *(Privada)*	**Lectura.**	No	**Apertura** y **Cierre**
`writeBlockOnDisk` *(Privada)*	**Escritura.**	No	No
`writeOnDisk` *(Privada)*	**Escritura.**	**Sí**	**Apertura** y **Cierre**
`fOpen` *(Privada)*	Nada	No	**Apertura**
`fClose` *(Privada)*	Nada	No	**Cierre**
`fCloseAndOpen` *(Privada)*	Nada	No	**Cierre** y **Apertura**
`addRecordOnHeader` *(Privada)*	No	No	No
`addBlockOnHeader` *(Privada)*	No	No	No
`getNonDeleted` *(Privada)*	No	No	No
`copyOnFile` *(Privada)*	**Escritura** *en otro fichero*	**Sí** *en el otro fichero al añadir*	**Apertura** *del otro fichero*
`chargeNextBlock` *(Privada)*	**Lectura** y **Escritura**	No	No
`goToLastBlock` *(Privada)*	**Lectura** y **Escritura**	No	No
`findRecord` *(Privada)*	**Lectura**	No	**Apertura** y **Cierre**
`deleteRecordOnHeader` *(Privada)*	Nada	No	No

`Constructor` *(Pública)*	**Lectura**	**Sí** *al crear el archivo cuando sea necesario*	**Apertura**
`Destructor` *(Pública)*	**Escritura**	No	**Cierre**
`getRecord` *(Pública)*	**Lectura**	No	**Apertura** y **Cierre**
`reorder` *(Pública)*	**Lectura** y **Escritura** *(en el nuevo fichero)*	**Sí**	**Apertura** y **Cierre**
`insertRecord` *(Pública)*	**Escritura**	**Sí**	No
`deleteRecord` *(Pública)*	**Lectura** y **Escritura**	**Sí**	**Cierre** y **Apertura** *para buscar*
`modifyRecord` *(Pública)*	**Lectura** y **Escritura**	**Sí**	**Cierre** y **Apertura** *para buscar*

Cuadro 1: Resumen de operaciones físicas y de acceso a disco según el requerimiento del enunciado.